Practical Workbook

CT-361

Artificial Intelligence & Expert Systems





Name: Moazzam Farooqui

Roll No: CT-24068

Batch: 2024

Department: CSIT

Department of Computer Science & Information Technology
NED University of Engineering & Technology

LAB # 07 & 08: A * SEARCH, HILL CLIMBING & GENETIC ALGORITHM

1. A * SEARCH

QUESTION NO # 01

9. A* Lab Task

Implement A* for a city navigation system using the following weighted graph:

```
A → B = 4
A → C = 2
B → D = 5
C → D = 8
C → E = 10
D → E = 2
E → G = 3
```

Heuristic values:

```
A = 10
B = 7
C = 8
D = 5
E = 3
G = 0
```

Find the path A → G and display the selected node, g(n), h(n), f(n), parent, final path, and total cost.

SOLUTION:

```python
import heapq, math

def astar(graph, h, start, goal, trace=False):
    g = {start: 0}; parent = {start: None}
    open_ = [(h[start], start)]; expanded = 0; step = 0
    if trace:
        print(f"{'Step':<5}{'Selected':<10}{'g(n)':<7}{'h(n)':<7}{'f(n)':<7}{'Parent':<8}OPEN after expansion")
    while open_:
        f, cur = heapq.heappop(open_)
        if f > g[cur] + h[cur] + 1e-9:
            continue
        step += 1; expanded += 1
        if cur == goal:
            if trace:
                print(f"{step:<5}{cur:<10}{g[cur]:<7}{h[cur]:<7}{f:<7}{str(parent[cur]):<8}GOAL REACHED")
            path, n = [], cur
            while n is not None:
                path.append(n); n = parent[n]
            return path[::-1], g[cur], expanded
        for nb, w in graph.get(cur, []):
            tg = g[cur] + w
            if nb not in g or tg < g[nb]:
                g[nb] = tg; parent[nb] = cur
                heapq.heappush(open_, (tg + h[nb], nb))
        if trace:
            live = sorted((round(ff, 1), n) for ff, n in open_ if abs(ff - (g[n] + h[n])) < 1e-9)
            print(f"{step:<5}{cur:<10}{g[cur]:<7}{h[cur]:<7}{f:<7}{str(parent[cur]):<8}"
                  + ", ".join(f"{n}(f={ff})" for ff, n in live))
    return None, math.inf, expanded
```

```python
def build(edges, directed=False):
    gr = {}
    for u, v, w in edges:
        gr.setdefault(u, []).append((v, w)); gr.setdefault(v, [])
        if not directed: gr[v].append((u, w))
    return gr

def euclid(a, b): return math.hypot(a[0]-b[0], a[1]-b[1])

edges = [("A","B",4),("A","C",2),("B","D",5),("C","D",8),("C","E",10),("D","E",2),("E","G",3)]
H = {"A":10,"B":7,"C":8,"D":5,"E":3,"G":0}
path, cost, exp = astar(build(edges, True), H, "A", "G", trace=True)
print("\nFinal path :", " -> ".join(path)); print("Total cost :", cost); print("Nodes expanded:", exp)

def road_demo(title, coords, roads, start, goal, unit, trace=False):
    pos = coords
    edges = [(u, v, math.ceil(euclid(pos[u], pos[v])*m*10)/10) for u, v, m in roads]
    gr = build(edges)
    h = {n: round(euclid(pos[n], pos[goal]), 1) for n in pos}
    path, cost, exp = astar(gr, h, start, goal, trace)
    print(f"Route  : {' -> '.join(path)}"); print(f"Total distance : {cost:.1f} {unit}"); print(f"Nodes expanded : {exp}")
    return edges, h

```

OUTPUT:

```
Step Selected  g(n)   h(n)   f(n)   Parent  OPEN after expansion
1    A         0      10     10     None    C(f=10), B(f=11)
2    C         2      8      10     A       B(f=11), D(f=15), E(f=15)
3    B         4      7      11     A       D(f=14), E(f=15)
4    D         9      5      14     B       E(f=14)
5    E         11     3      14     D       G(f=14)
6    G         14     0      14     E       GOAL REACHED

Final path : A -> B -> D -> E -> G
Total cost : 14
Nodes expanded: 6
```

QUESTION NO # 02

10. Scenario-Based A* Problems

Scenario 1 — Hospital Emergency Route

An ambulance needs to travel from Hospital H to Emergency Location E. Intersections are represented as nodes and roads have different distances.

Task: Implement A* to find the shortest route. Use road distance as g(n) and straight-line distance as h(n). Display the path and total distance.

SOLUTION:

```
# Scenario 1: Hospital Emergency Route

coords = {"H":(0,0),"J1":(2,1),"J2":(1,3),"J3":(4,0),"J4":(4,3),"J5":(6,1),"J6":(6,4),"ER":(8,3)}
roads = [("H","J1",1.1),("H","J2",1.3),("J1","J2",1.2),("J1","J3",1.2),("J1","J4",1.4),("J2","J4",1.3),
         ("J3","J4",1.5),("J3","J5",1.1),("J4","J5",1.3),("J4","J6",1.2),("J5","ER",1.2),("J6","ER",1.1)]
edges1, h1 = road_demo("S1", coords, roads, "H", "ER", "km", trace=True)
print("Road distances (g):", {f"{u}-{v}": w for u, v, w in edges1})
print("Straight-line h(n) to ER:", h1)
```

OUTPUT:

```
Final path : A -> B -> D -> E -> G
Total cost : 14
Nodes expanded: 6
Step Selected  g(n)   h(n)   f(n)   Parent  OPEN after expansion
1    H         0      8.5    8.5    None    J1(f=8.8), J2(f=11.2)
2    J1        2.5    6.3    8.8    H       J3(f=10.2), J4(f=10.5), J2(f=11.2)
3    J3        5.2    5.0    10.2   J1      J4(f=10.5), J5(f=10.5), J2(f=11.2)
4    J4        6.5    4.0    10.5   J1      J5(f=10.5), J2(f=11.2), J6(f=11.4)
5    J5        7.7    2.8    10.5   J3      ER(f=11.1), J2(f=11.2), J6(f=11.4)
6    ER        11.1   0.0    11.1   J5      GOAL REACHED
Route  : H -> J1 -> J3 -> J5 -> ER
Total distance : 11.1 km
Nodes expanded : 6
Road distances (g): {'H-J1': 2.5, 'H-J2': 4.2, 'J1-J2': 2.7, 'J1-J3': 2.7, 'J1-J4': 4.0, 'J2-J4': 3.9, 'J3-J4'
: 4.5, 'J3-J5': 2.5, 'J4-J5': 3.7, 'J4-J6': 2.7, 'J5-ER': 3.4, 'J6-ER': 2.5}
Straight-line h(n) to ER: {'H': 8.5, 'J1': 6.3, 'J2': 7.0, 'J3': 5.0, 'J4': 4.0, 'J5': 2.8, 'J6': 2.2, 'ER': 0
.0}
```

QUESTION NO # 03

Scenario 2 — Delivery Robot

An autonomous delivery robot moves through a warehouse represented as a grid. S is the starting point, P is the package location, and D is the delivery point. Obstacles block some cells.

Task: Use A* to find the shortest route from S to D. Use Manhattan distance as the heuristic:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

Students must create a grid, mark obstacles, assign movement costs, calculate heuristic values, and find the shortest route.

SOLUTION:

```python
# Scenario 2: Delivery Robot (Grid)

GRID = ["S..#....",
        ".#.#.##.",
        "##...#..",
        ".#.#.#.#",
        "..P.~~~.",
        "####.##.",
        "....~..D"]
# legend: # obstacle, . cost 1, ~ cost 3 (slippery floor), S/P/D cost 1
R, C = len(GRID), len(GRID[0])
find = lambda ch: next((r, c) for r in range(R) for c in range(C) if GRID[r][c] == ch)
cost_of = lambda r, c: 3 if GRID[r][c] == "~" else 1
def grid_graph():
    gr = {}
    for r in range(R):
        for c in range(C):
            if GRID[r][c] == "#": continue
            gr[(r, c)] = [((r+dr, c+dc), cost_of(r+dr, c+dc)) for dr, dc in ((1,0),(-1,0),(0,1),(0,-1))
                          if 0 <= r+dr < R and 0 <= c+dc < C and GRID[r+dr][c+dc] != "#"]
    return gr
gg = grid_graph(); S, P, D = find("S"), find("P"), find("D")
def leg(a, b, name):
    h = {n: abs(n[0]-b[0]) + abs(n[1]-b[1]) for n in gg}
    p, c, e = astar(gg, h, a, b)
    print(f"\nLeg {name}: start={a}, goal={b}")
    print("Manhattan heuristic grid (X = obstacle):")
    for r in range(R): print("  " + " ".join(f"{h[(r,cc)]:>2}" if (r,cc) in h else " X" for cc in range(C)))
    print(f"Path: {p}\nCost: {c}   Nodes expanded: {e}")
    return p, c
p1, c1 = leg(S, P, "S -> P"); p2, c2 = leg(P, D, "P -> D")
print("\nGrid with route (* = path):")
full = set(p1 + p2)
for r in range(R): print("  " + " ".join("*" if (r,c) in full and GRID[r][c] not in "SPD" else GRID[r][c] for c in range(C)))
print(f"\nTotal route cost S -> P -> D = {c1} + {c2} = {c1+c2}")
```

OUTPUT:

```
Leg S -> P: start=(0, 0), goal=(4, 2)
Manhattan heuristic grid (X = obstacle):
   6  5  4  X  6  7  8  9
   5  X  3  X  5  X  X  8
   X  X  2  3  4  X  6  7
   3  X  1  X  3  X  5  X
   2  1  0  1  2  3  4  5
   X  X  X  X  3  X  X  6
   4  3  2  3  4  5  6  7
Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (4, 2)]
Cost: 6   Nodes expanded: 8

Leg P -> D: start=(4, 2), goal=(6, 7)
Manhattan heuristic grid (X = obstacle):
  13 12 11  X  9  8  7  6
  12  X 10  X  8  X  X  5
   X  X  9  8  7  X  5  4
  10  X  8  X  6  X  4  X
   9  8  7  6  5  4  3  2
   X  X  X  X  4  X  X  1
   7  6  5  4  3  2  1  0
Path: [(4, 2), (4, 3), (4, 4), (5, 4), (6, 4), (6, 5), (6, 6), (6, 7)]
Cost: 11   Nodes expanded: 16

Grid with route (* = path):
  S * * # . . . .
  . # * # . # # .
  # # * . . # . .
  . # * # . # . #
  . . P * * ~ ~ .
  # # # # * # # .
  . . . . * * * D

Total route cost S -> P -> D = 6 + 11 = 17
```

QUESTION NO # 04

Scenario 3 — University Campus Navigation

A university contains Main Gate, Library, CS Department, Cafeteria, Lab, and Administration. Roads between buildings have different distances.

Task: Develop an A* navigation system that accepts a starting building and destination building and returns the shortest route and total distance.

SOLUTION:

```
# Scenario 3: University Campus Navigation

cc = {"Main Gate":(0,0),"Library":(200,150),"Administration":(150,-120),"Cafeteria":(350,0),
      "CS Department":(450,200),"Lab":(600,100)}
cr = [("Main Gate","Library",1.1),("Main Gate","Administration",1.2),("Library","Cafeteria",1.3),
      ("Library","CS Department",1.2),("Administration","Cafeteria",1.1),("Cafeteria","CS Department",1.4),
      ("Cafeteria","Lab",1.2),("CS Department","Lab",1.1)]
cedges = [(u, v, math.ceil(euclid(cc[u], cc[v])*m)) for u, v, m in cr]; cgraph = build(cedges)
def navigate(a, b):
    h = {n: euclid(cc[n], cc[b]) for n in cc}
    p, c, e = astar(cgraph, h, a, b)
    print(f"{a} -> {b}: {' -> '.join(p)} | {c:.0f} m | expanded {e}")
print("Buildings:", list(cc)); print("Roads (m):", {f"{u}-{v}": w for u, v, w in cedges}); print()
for a, b in [("Main Gate","Lab"),("Administration","CS Department"),("Library","Cafeteria"),("Lab","Main Gate")]:
    navigate(a, b)
```

OUTPUT:

```
Buildings: ['Main Gate', 'Library', 'Administration', 'Cafeteria', 'CS Department', 'Lab']
Roads (m): {'Main Gate-Library': 275, 'Main Gate-Administration': 231, 'Library-Cafeteria': 276, 'Library-CS
epartment': 306, 'Administration-Cafeteria': 257, 'Cafeteria-CS Department': 314, 'Cafeteria-Lab': 324, 'CS D
partment-Lab': 199}

Main Gate -> Lab: Main Gate -> Library -> CS Department -> Lab | 780 m | expanded 6
Administration -> CS Department: Administration -> Cafeteria -> CS Department | 571 m | expanded 3
Library -> Cafeteria: Library -> Cafeteria | 276 m | expanded 2
Lab -> Main Gate: Lab -> CS Department -> Library -> Main Gate | 780 m | expanded 6
```

2. HILL CLIMBING ALGORITHM

QUESTION NO # 01

9. Hill Climbing Lab Task

Implement a program that maximizes:

$$f(x) = x^2 + 2x, \quad 0 \leq x \leq 15$$

10. Select an initial value.
11. Generate neighboring values.
12. Calculate the fitness.
13. Select a better neighbor.
14. Continue until no improvement is possible.
15. Display every iteration and the final solution.

SOLUTION:

```python
import random, itertools, math

f = lambda x: x*x + 2*x
def neighbors(x): return [n for n in (x-1, x+1) if 0 <= n <= 15]

def steepest(x):
    it = 0
    while True:
        nb = {n: f(n) for n in neighbors(x)}
        best = max(nb, key=nb.get)
        print(f"Iter {it}: x={x:<2} f={f(x):<4} neighbors={nb} -> best neighbor x={best} (f={nb[best]})", end="")
        if nb[best] <= f(x):
            print("  => no better neighbor, STOP"); return x
        print("  => MOVE"); x = best; it += 1

def simple(x):
    it = 0
    while True:
        moved = False
        for n in neighbors(x):
            if f(n) > f(x):
                print(f"Iter {it}: x={x:<2} f={f(x):<4} first better neighbor x={n} (f={f(n)}) => MOVE")
                x = n; moved = True; it += 1; break
        if not moved:
            print(f"Iter {it}: x={x} f={f(x)} no better neighbor => STOP"); return x

print("\n-- Steepest-ascent, initial x = 3 --"); x = steepest(3)
print(f"FINAL: x={x}, f(x)={f(x)}")
print("\n-- Simple hill climbing, initial x = 9 --"); x = simple(9)
print(f"FINAL: x={x}, f(x)={f(x)}")
```

OUTPUT:

```
-- Steepest-ascent, initial x = 3 --
Iter 0: x=3  f=15   neighbors={2: 8, 4: 24} -> best neighbor x=4 (f=24)  => MOVE
Iter 1: x=4  f=24   neighbors={3: 15, 5: 35} -> best neighbor x=5 (f=35)  => MOVE
Iter 2: x=5  f=35   neighbors={4: 24, 6: 48} -> best neighbor x=6 (f=48)  => MOVE
Iter 3: x=6  f=48   neighbors={5: 35, 7: 63} -> best neighbor x=7 (f=63)  => MOVE
Iter 4: x=7  f=63   neighbors={6: 48, 8: 80} -> best neighbor x=8 (f=80)  => MOVE
Iter 5: x=8  f=80   neighbors={7: 63, 9: 99} -> best neighbor x=9 (f=99)  => MOVE
Iter 6: x=9  f=99   neighbors={8: 80, 10: 120} -> best neighbor x=10 (f=120)  => MOVE
Iter 7: x=10 f=120  neighbors={9: 99, 11: 143} -> best neighbor x=11 (f=143)  => MOVE
Iter 8: x=11 f=143  neighbors={10: 120, 12: 168} -> best neighbor x=12 (f=168)  => MOVE
Iter 9: x=12 f=168  neighbors={11: 143, 13: 195} -> best neighbor x=13 (f=195)  => MOVE
Iter 10: x=13 f=195  neighbors={12: 168, 14: 224} -> best neighbor x=14 (f=224)  => MOVE
Iter 11: x=14 f=224  neighbors={13: 195, 15: 255} -> best neighbor x=15 (f=255)  => MOVE
Iter 12: x=15 f=255  neighbors={14: 224} -> best neighbor x=14 (f=224)  => no better neighbor, STOP
FINAL: x=15, f(x)=255

-- Simple hill climbing, initial x = 9 --
Iter 0: x=9  f=99   first better neighbor x=10 (f=120) => MOVE
Iter 1: x=10 f=120  first better neighbor x=11 (f=143) => MOVE
Iter 2: x=11 f=143  first better neighbor x=12 (f=168) => MOVE
Iter 3: x=12 f=168  first better neighbor x=13 (f=195) => MOVE
Iter 4: x=13 f=195  first better neighbor x=14 (f=224) => MOVE
Iter 5: x=14 f=224  first better neighbor x=15 (f=255) => MOVE
Iter 6: x=15 f=255 no better neighbor => STOP
FINAL: x=15, f(x)=255
```

QUESTION NO # 02

10. Scenario-Based Hill Climbing Problems

Scenario 1 — Employee Shift Scheduling

A hospital wants to create an efficient staff schedule. Each schedule receives a fitness score based on patients covered, doctors available, overtime penalties, and staff availability.

Task: Use hill climbing to improve an initial schedule. Generate a neighbor by changing one employee's shift. Continue until no neighboring schedule has a higher fitness.

SOLUTION:

```python
#  Scenario 1: Employee Shift Scheduling

random.seed(11)
SH = ["Morning","Evening","Night","OFF"]; DAYS = 3
EMP = {"Dr.Ali":("doctor",{(0,2)}),"Dr.Sara":("doctor",{(1,0)}),"Dr.Omar":("doctor",set()),
       "N.Hina":("nurse",set()),"N.Zain":("nurse",{(2,1)}),"N.Ayesha":("nurse",set()),
       "N.Bilal":("nurse",{(0,0)}),"N.Fatima":("nurse",set())}
names = list(EMP)
def sfit(s):
    t = 0
    for d in range(DAYS):
        for sh in range(3):
            who = [e for e in names if s[e][d] == sh]
            dc = sum(EMP[e][0] == "doctor" for e in who)
            t += 10*min(len(who), 2) + (15 if dc >= 1 else 0) - 5*max(0, len(who)-2)   # coverage rewards
    for e in names:
        t -= 25*sum((d, s[e][d]) in EMP[e][1] for d in range(DAYS))                    # unavailable shift
        t -= 12*max(0, sum(s[e][d] != 3 for d in range(DAYS)) - 2)                     # overtime (>2 shifts)
    return t
def show(s):
    print(f"{'Employee':<10}" + "".join(f"Day{d+1:<9}" for d in range(DAYS)))
    for e in names: print(f"{e:<10}" + "".join(f"{SH[s[e][d]]:<12}" for d in range(DAYS)))
sched = {e: [random.randrange(4) for _ in range(DAYS)] for e in names}
print("Max possible coverage score = 9 slots x 35 = 315\nInitial schedule, fitness =", sfit(sched)); show(sched)
it = 0
while True:
    best, bf = None, sfit(sched)
    for e in names:
        for d in range(DAYS):
            for sh in range(4):
                if sh != sched[e][d]:
                    old = sched[e][d]; sched[e][d] = sh; v = sfit(sched); sched[e][d] = old
                    if v > bf: bf, best = v, (e, d, sh)
    if best is None: break
    e, d, sh = best; it += 1
    print(f"Iter {it}: change {e} on Day{d+1}: {SH[sched[e][d]]} -> {SH[sh]}  fitness -> {bf}")
    sched[e][d] = sh
print(f"No neighbour has higher fitness. FINAL fitness = {sfit(sched)}"); show(sched)
```

OUTPUT:

```
Initial schedule, fitness = 84
Employee  Day1       Day2       Day3
Dr.Ali    OFF        OFF        OFF
Dr.Sara   Evening    Evening    OFF
Dr.Omar   Evening    Morning    OFF
N.Hina    Night      Evening    Morning
N.Zain    Morning    OFF        OFF
N.Ayesha  Evening    Morning    Morning
N.Bilal   Morning    Morning    Evening
N.Fatima  Evening    Morning    OFF
Iter 1: change Dr.Sara on Day1: Evening -> Night  fitness -> 114
Iter 2: change N.Bilal on Day1: Morning -> OFF  fitness -> 141
Iter 3: change Dr.Ali on Day1: OFF -> Morning  fitness -> 166
Iter 4: change Dr.Ali on Day2: OFF -> Night  fitness -> 191
Iter 5: change N.Ayesha on Day1: Evening -> OFF  fitness -> 208
Iter 6: change N.Ayesha on Day2: Morning -> Night  fitness -> 223
Iter 7: change Dr.Ali on Day3: OFF -> Evening  fitness -> 236
Iter 8: change Dr.Sara on Day3: OFF -> Night  fitness -> 249
Iter 9: change N.Zain on Day3: OFF -> Night  fitness -> 259
Iter 10: change N.Bilal on Day2: Morning -> OFF  fitness -> 264
Iter 11: change N.Hina on Day1: Night -> OFF  fitness -> 266
Iter 12: change N.Bilal on Day1: OFF -> Night  fitness -> 276
No neighbour has higher fitness. FINAL fitness = 276
Employee  Day1       Day2       Day3
Dr.Ali    Morning    Night      Evening
Dr.Sara   Night      Evening    Night
Dr.Omar   Evening    Morning    OFF
N.Hina    OFF        Evening    Morning
N.Zain    Morning    OFF        Night
N.Ayesha  OFF        Night      Morning
N.Bilal   Night      OFF        Evening
N.Fatima  Evening    Morning    OFF
```

QUESTION NO # 03

Scenario 2 — Delivery Route Optimization

A delivery company has several possible routes. Fitness is defined as Fitness = 1 / Total Distance. A neighboring route is generated by swapping two delivery locations.

Task: Implement hill climbing to improve the route. Display the initial route, initial fitness, neighbor routes, selected route, final route, and final fitness.

SOLUTION:

```python
# Scenario 2: Delivery Route Optimization

random.seed(3)
pts = {"Depot":(25,25), **{f"L{i}":(random.randint(0,50), random.randint(0,50)) for i in range(1,8)}}
dist = lambda a, b: math.dist(pts[a], pts[b])
def tour(r): p = ["Depot"]+list(r)+["Depot"]; return sum(dist(p[i], p[i+1]) for i in range(len(p)-1))
fit = lambda r: 1/tour(r)
route = [f"L{i}" for i in range(1, 8)]; random.shuffle(route)
print("Locations:", pts); print(f"Initial route : {route}\nInitial distance={tour(route):.2f}  fitness={fit(route):.6f}")
it = 0
while True:
    nbrs = []
    for i, j in itertools.combinations(range(7), 2):
        n = route[:]; n[i], n[j] = n[j], n[i]; nbrs.append((fit(n), n, (i, j)))
    best = max(nbrs)
    if it == 0:
        print("\nNeighbour routes of iteration 1 (swap positions i,j):")
        for fv, n, ij in nbrs: print(f"  swap{ij}: {n}  dist={1/fv:.2f}  fitness={fv:.6f}")
    if best[0] <= fit(route): print(f"Iter {it+1}: no better neighbour -> STOP"); break
    it += 1; route = best[1]
    print(f"Iter {it}: selected swap{best[2]} -> {route}  dist={tour(route):.2f}  fitness={fit(route):.6f}")
print(f"\nFinal route: Depot -> {' -> '.join(route)} -> Depot\nFinal distance={tour(route):.2f}  fitness={fit(route):.6f}")
opt = min(itertools.permutations(route), key=tour); print(f"(Brute-force optimum = {tour(opt):.2f})")
```

OUTPUT:

```
Locations: {'Depot': (25, 25), 'L1': (15, 37), 'L2': (34, 8), 'L3': (23, 38), 'L4': (30, 40), 'L5': (37, 4), 'L6': (38, 0), 'L7': (30, 16)}
Initial route : ['L1', 'L7', 'L3', 'L4', 'L6', 'L2', 'L5']
Initial distance=150.72  fitness=0.006635

Neighbour routes of iteration 1 (swap positions i,j):
  swap(0, 1): ['L7', 'L1', 'L3', 'L4', 'L6', 'L2', 'L5']  dist=130.37  fitness=0.007671
  swap(0, 2): ['L3', 'L7', 'L1', 'L4', 'L6', 'L2', 'L5']  dist=156.27  fitness=0.006399
  swap(0, 3): ['L4', 'L7', 'L3', 'L1', 'L6', 'L2', 'L5']  dist=152.66  fitness=0.006551
  swap(0, 4): ['L6', 'L7', 'L3', 'L4', 'L1', 'L2', 'L5']  dist=155.59  fitness=0.006427
  swap(0, 5): ['L2', 'L7', 'L3', 'L4', 'L6', 'L1', 'L5']  dist=206.75  fitness=0.004837
  swap(0, 6): ['L5', 'L7', 'L3', 'L4', 'L6', 'L2', 'L1']  dist=168.47  fitness=0.005936
  swap(1, 2): ['L1', 'L3', 'L7', 'L4', 'L6', 'L2', 'L5']  dist=149.69  fitness=0.006680
  swap(1, 3): ['L1', 'L4', 'L3', 'L7', 'L6', 'L2', 'L5']  dist=117.30  fitness=0.008525
  swap(1, 4): ['L1', 'L6', 'L3', 'L4', 'L7', 'L2', 'L5']  dist=169.45  fitness=0.005901
  swap(1, 5): ['L1', 'L2', 'L3', 'L4', 'L6', 'L7', 'L5']  dist=186.28  fitness=0.005368
  swap(1, 6): ['L1', 'L5', 'L3', 'L4', 'L6', 'L2', 'L7']  dist=168.31  fitness=0.005942
  swap(2, 3): ['L1', 'L7', 'L4', 'L3', 'L6', 'L2', 'L5']  dist=151.69  fitness=0.006592
  swap(2, 4): ['L1', 'L7', 'L6', 'L4', 'L3', 'L2', 'L5']  dist=168.53  fitness=0.005934
  swap(2, 5): ['L1', 'L7', 'L2', 'L4', 'L6', 'L3', 'L5']  dist=225.22  fitness=0.004440
  swap(2, 6): ['L1', 'L7', 'L5', 'L4', 'L6', 'L2', 'L3']  dist=186.84  fitness=0.005352
  swap(3, 4): ['L1', 'L7', 'L3', 'L6', 'L4', 'L2', 'L5']  dist=207.60  fitness=0.004817
  swap(3, 5): ['L1', 'L7', 'L3', 'L2', 'L6', 'L4', 'L5']  dist=207.06  fitness=0.004829
  swap(3, 6): ['L1', 'L7', 'L3', 'L5', 'L6', 'L2', 'L4']  dist=162.41  fitness=0.006157
  swap(4, 5): ['L1', 'L7', 'L3', 'L4', 'L2', 'L6', 'L5']  dist=141.30  fitness=0.007077
  swap(4, 6): ['L1', 'L7', 'L3', 'L4', 'L5', 'L2', 'L6']  dist=150.59  fitness=0.006641
  swap(5, 6): ['L1', 'L7', 'L3', 'L4', 'L6', 'L5', 'L2']  dist=140.95  fitness=0.007095
Iter 1: selected swap(1, 3) -> ['L1', 'L4', 'L3', 'L7', 'L6', 'L2', 'L5']  dist=117.30  fitness=0.008525
Iter 2: selected swap(3, 6) -> ['L1', 'L4', 'L3', 'L5', 'L6', 'L2', 'L7']  dist=107.27  fitness=0.009322
Iter 3: selected swap(1, 2) -> ['L1', 'L3', 'L4', 'L5', 'L6', 'L2', 'L7']  dist=99.94  fitness=0.010006
Iter 4: no better neighbour -> STOP

Final route: Depot -> L1 -> L3 -> L4 -> L5 -> L6 -> L2 -> L7 -> Depot
Final distance=99.94  fitness=0.010006
(Brute-force optimum = 99.94)
```

QUESTION NO # 04

Scenario 3 — Exam Timetable Optimization

A university wants to generate an examination timetable. The fitness function rewards conflict-free scheduling, room availability, and teacher availability while penalizing student, room, and teacher conflicts.

Task: Represent a timetable as a solution. Generate neighboring timetables by changing one exam's time slot. Use hill climbing to maximize fitness.

SOLUTION:

```python
import random, itertools

EXAMS = {
    "E1": ("T1", {"G1", "G2"}, "R1"),
    "E2": ("T2", {"G1"},       "R2"),
    "E3": ("T1", {"G3"},       "R1"),
    "E4": ("T3", {"G2", "G3"}, "R2"),
    "E5": ("T2", {"G4"},       "R1"),
    "E6": ("T3", {"G4", "G1"}, "R3"),
}
SLOTS = ["Mon 9am", "Mon 2pm", "Tue 9am", "Tue 2pm"]
TEACHER_UNAVAILABLE = {"T1": {3}, "T3": {0}}      # slot indices a teacher cannot attend
ROOM_UNAVAILABLE    = {"R2": {1}, "R3": {2}}      # slot indices a room is not free

REWARD_FREE, REWARD_TEACHER_OK, REWARD_ROOM_OK = 10, 5, 5
PEN_STUDENT, PEN_ROOM, PEN_TEACHER = 20, 15, 15
MAX_FITNESS = len(EXAMS) * (REWARD_FREE + REWARD_TEACHER_OK + REWARD_ROOM_OK)   # 120

def evaluate(t):
    """t = timetable: dict exam -> slot index. Returns (fitness, details)."""
    stu = rm = tea = 0
    in_conflict = set()
    for a, b in itertools.combinations(EXAMS, 2):
        if t[a] == t[b]:
            if EXAMS[a][1] & EXAMS[b][1]: stu += 1; in_conflict |= {a, b}
            if EXAMS[a][2] == EXAMS[b][2]: rm += 1;  in_conflict |= {a, b}
            if EXAMS[a][0] == EXAMS[b][0]: tea += 1; in_conflict |= {a, b}
    free   = len(EXAMS) - len(in_conflict)                                  # conflict-free exams
    t_ok   = sum(t[e] not in TEACHER_UNAVAILABLE.get(EXAMS[e][0], set()) for e in EXAMS)
    r_ok   = sum(t[e] not in ROOM_UNAVAILABLE.get(EXAMS[e][2], set()) for e in EXAMS)
    fit = (REWARD_FREE*free + REWARD_TEACHER_OK*t_ok + REWARD_ROOM_OK*r_ok
           - PEN_STUDENT*stu - PEN_ROOM*rm - PEN_TEACHER*tea)
    return fit, dict(free=free, teacher_ok=t_ok, room_ok=r_ok, student=stu, room=rm, teacher=tea)

def fitness(t): return evaluate(t)[0]
```

```python
def neighbours(t):
    """Every timetable obtained by changing ONE exam's time slot."""
    for e in EXAMS:
        for s in range(len(SLOTS)):
            if s != t[e]:
                n = dict(t); n[e] = s
                yield (e, s), n

def show(t):
    for i, name in enumerate(SLOTS):
        items = [f"{e}({EXAMS[e][0]},{EXAMS[e][2]})" for e in EXAMS if t[e] == i]
        print(f"   {name:<8}: {', '.join(items) if items else '-'}")
    f, d = evaluate(t)
    print(f"   fitness={f}/{MAX_FITNESS} | conflict-free exams={d['free']}, teacher-available={d['teacher_ok']}, "
          f"room-available={d['room_ok']} | student conflicts={d['student']}, room conflicts={d['room']}, "
          f"teacher conflicts={d['teacher']}")

def hill_climb(start, verbose=True):
    t, it = dict(start), 0
    while True:
        nb = list(neighbours(t))
        if verbose and it == 0:
            print(f"\n   Iteration 1 considers {len(nb)} neighbours, e.g.:")
            for (e, s), n in nb[:6]:
                print(f"     change {e} to {SLOTS[s]:<8} -> fitness {fitness(n)}")
        (move, best) = max(nb, key=lambda x: fitness(x[1]))
        if fitness(best) <= fitness(t):
            if verbose: print(f"Iteration {it+1}: best neighbour fitness {fitness(best)} <= current {fitness(t)} -> STOP (local/global maximum)")
            return t, it
        it += 1
        if verbose: print(f"Iteration {it}: move {move[0]} {SLOTS[t[move[0]]]} -> {SLOTS[move[1]]}   fitness {fitness(t)} -> {fitness(best)}")
        t = best
```

```python
print("Solution representation : dictionary  exam -> time-slot index")
print("Neighbour               : same timetable with ONE exam moved to a different slot")
print(f"Fitness (max {MAX_FITNESS})        : +{REWARD_FREE} per conflict-free exam, +{REWARD_TEACHER_OK} per exam in a teacher-available slot,"
      f" +{REWARD_ROOM_OK} per exam in a room-available slot,")
print(f"                          -{PEN_STUDENT} per student conflict, -{PEN_ROOM} per room conflict, -{PEN_TEACHER} per teacher conflict")
print("Teacher unavailable     :", {k: [SLOTS[i] for i in v] for k, v in TEACHER_UNAVAILABLE.items()})
print("Room unavailable        :", {k: [SLOTS[i] for i in v] for k, v in ROOM_UNAVAILABLE.items()})

random.seed(4)
start = {e: random.randrange(len(SLOTS)) for e in EXAMS}
print("\nINITIAL TIMETABLE"); show(start)
final, its = hill_climb(start)
print(f"\nFINAL TIMETABLE after {its} moves"); show(final)

best, bestf, results = None, -10**9, []
for seed in range(20):
    random.seed(seed); s0 = {e: random.randrange(len(SLOTS)) for e in EXAMS}
    r, _ = hill_climb(s0, verbose=False); results.append(fitness(r))
    if fitness(r) > bestf: best, bestf = r, fitness(r)
print(f"\nRandom restarts (20 runs): final fitness values = {sorted(results)}")
print(f"Runs reaching the maximum {max(results)}: {results.count(max(results))}/20")
print("\nBEST TIMETABLE OVER ALL RESTARTS"); show(best)
```

OUTPUT:

```
Solution representation : dictionary  exam -> time-slot index
Neighbour               : same timetable with ONE exam moved to a different slot
Fitness (max 120)        : +10 per conflict-free exam, +5 per exam in a teacher-available slot, +5 per exam in a room-available slot,
                           -20 per student conflict, -15 per room conflict, -15 per teacher conflict
Teacher unavailable     : {'T1': ['Tue 2pm'], 'T3': ['Mon 9am']}
Room unavailable        : {'R2': ['Mon 2pm'], 'R3': ['Tue 9am']}

INITIAL TIMETABLE
   Mon 9am : E3(T1,R1)
   Mon 2pm : E1(T1,R1), E6(T3,R3)
   Tue 9am : E2(T2,R2)
   Tue 2pm : E4(T3,R2), E5(T2,R1)
   fitness=80/120 | conflict-free exams=4, teacher-available=6, room-available=6 | student conflicts=1, room conflicts=0, teacher conflicts=0

   Iteration 1 considers 18 neighbours, e.g.:
     change E1 to Mon 9am  -> fitness 70
     change E1 to Tue 9am  -> fitness 80
     change E1 to Tue 2pm  -> fitness 50
     change E2 to Mon 9am  -> fitness 80
     change E2 to Mon 2pm  -> fitness 25
     change E2 to Tue 2pm  -> fitness 20
Iteration 1: move E6 Mon 2pm -> Mon 9am   fitness 80 -> 115
Iteration 2: best neighbour fitness 115 <= current 115 -> STOP (local/global maximum)

FINAL TIMETABLE after 1 moves
   Mon 9am : E3(T1,R1), E6(T3,R3)
   Mon 2pm : E1(T1,R1)
   Tue 9am : E2(T2,R2)
   Tue 2pm : E4(T3,R2), E5(T2,R1)
   fitness=115/120 | conflict-free exams=6, teacher-available=5, room-available=6 | student conflicts=0, room conflicts=0, teacher conflicts=0

Random restarts (20 runs): final fitness values = [105, 105, 105, 105, 110, 110, 115, 115, 115, 115, 115, 115, 115, 115, 115, 115, 120, 120, 120, 120]
Runs reaching the maximum 120: 4/20

BEST TIMETABLE OVER ALL RESTARTS
   Mon 9am : E2(T2,R2), E3(T1,R1)
   Mon 2pm : E1(T1,R1)
   Tue 9am : E4(T3,R2), E5(T2,R1)
   Tue 2pm : E6(T3,R3)
   fitness=120/120 | conflict-free exams=6, teacher-available=6, room-available=6 | student conflicts=0, room conflicts=0, teacher conflicts=0
```

3. GENETIC ALGORITHM

QUESTION NO # 01

13. GA Lab Task

Implement a Genetic Algorithm to maximize:

$$f(x) = x^2 + 2x, \quad 0 \leq x \leq 15$$

Use the following configuration:

Parameter	Requirement
Chromosome length	4 bits
Population size	6
Selection	Tournament Selection
Crossover	One Point
Mutation	Bit Flip
Output	Generation-wise chromosomes, x, fitness, and best solution

SOLUTION:

```python
import random, itertools, math

random.seed(5)
L, POP, GENS, PC, PM = 4, 6, 8, 0.9, 0.08
x_of = lambda c: int(c, 2); fit = lambda c: x_of(c)**2 + 2*x_of(c)
def tournament(pop, k=2):
    return max(random.sample(pop, k), key=fit)
def crossover(a, b):
    if random.random() < PC:
        p = random.randint(1, L-1); return a[:p]+b[p:], b[:p]+a[p:], p
    return a, b, None
def mutate(c):
    return "".join(("1" if ch == "0" else "0") if random.random() < PM else ch for ch in c)
pop = ["".join(random.choice("01") for _ in range(L)) for _ in range(POP)]
def table(pop, g):
    print(f"\nGeneration {g}")
    print(f"  {'Chromosome':<12}{'x':<4}{'Fitness':<8}")
    for c in pop: print(f"  {c:<12}{x_of(c):<4}{fit(c):<8}")
    b = max(pop, key=fit); print(f"  Best: {b} (x={x_of(b)}, f={fit(b)})")
table(pop, 0); best_overall = max(pop, key=fit)
for g in range(1, GENS+1):
    new = [max(pop, key=fit)]
    while len(new) < POP:
        p1, p2 = tournament(pop), tournament(pop)
        c1, c2, _ = crossover(p1, p2); new += [mutate(c1), mutate(c2)]
    pop = new[:POP]; table(pop, g)
    best_overall = max(pop + [best_overall], key=fit)
print(f"\nBEST SOLUTION FOUND: chromosome={best_overall}, x={x_of(best_overall)}, f(x)={fit(best_overall)}  (true optimum: x=15, f=255)")
```

OUTPUT:

```
Generation 0
  Chromosome  x   Fitness
  1101        13  195
  0000        0   0
  1101        13  195
  0000        0   0
  1101        13  195
  0001        1   3
  Best: 1101 (x=13, f=195)

Generation 1
  Chromosome  x   Fitness
  1101        13  195
  0101        5   35
  1001        9   99
  0011        3   15
  1001        9   99
  1111        15  255
  Best: 1111 (x=15, f=255)

Generation 2
  Chromosome  x   Fitness
  1111        15  255
  0001        1   3
  1001        9   99
  1001        9   99
  1001        9   99
  1011        11  143
  Best: 1111 (x=15, f=255)

Generation 3
  Chromosome  x   Fitness
  1111        15  255
  1011        11  143
  1111        15  255
  1011        11  143
  1011        11  143
  1001        9   99
  Best: 1111 (x=15, f=255)
```

```
Generation 4
  Chromosome  x   Fitness
  1111        15  255
  1010        10  120
  1110        14  224
  1111        15  255
  1101        13  195
  1111        15  255
  Best: 1111 (x=15, f=255)

Generation 5
  Chromosome  x   Fitness
  1111        15  255
  1111        15  255
  1011        11  143
  1110        14  224
  1111        15  255
  1111        15  255
  Best: 1111 (x=15, f=255)

Generation 6
  Chromosome  x   Fitness
  1111        15  255
  1101        13  195
  1110        14  224
  1111        15  255
  1111        15  255
  1111        15  255
  Best: 1111 (x=15, f=255)

Generation 7
  Chromosome  x   Fitness
  1111        15  255
  1110        14  224
  1111        15  255
  0111        7   63
  1011        11  143
  1111        15  255
  Best: 1111 (x=15, f=255)
```

```
Generation 8
  Chromosome  x   Fitness
  1111        15  255
  1111        15  255
  0011        3   15
  1110        14  224
  1111        15  255
  1111        15  255
  Best: 1111 (x=15, f=255)

BEST SOLUTION FOUND: chromosome=1111, x=15, f(x)=255  (true optimum: x=15, f=255)
```

QUESTION NO # 02

14. Scenario-Based Genetic Algorithm Problems

Scenario 1 — Employee Selection

A company wants to select employees for a project. Each employee has a skill score, experience, availability, and salary requirement.

A chromosome such as 101101 indicates which employees are selected, where 1 means selected and 0 means not selected.

Task: Develop a GA that maximizes team quality while keeping salary below a specified budget. Reward high skill, experience, and availability, and penalize exceeding the budget.

SOLUTION:

```python
import random, itertools

def team_ga(emps, budget, min_size, max_size, min_senior, pop_n, gens, seed, verbose=False, pm=0.05):
    random.seed(seed); n = len(emps)
    def info(c):
        sel = [emps[i] for i in range(n) if c[i] == "1"]
        return sel, sum(e[4] for e in sel)
    def tfit(c):
        sel, sal = info(c)
        v = sum(3*e[1] + e[2] + 10*e[3] for e in sel)
        v -= 2*max(0, sal-budget)                              # over-budget penalty
        v -= 20*max(0, min_size-len(sel)) + 20*max(0, len(sel)-max_size)
        v -= 25*max(0, min_senior - sum(e[1] >= 8 for e in sel))
        return round(v, 1)
    def tour(pop, k=3): return max(random.sample(pop, k), key=tfit)
    pop = ["".join(random.choice("01") for _ in range(n)) for _ in range(pop_n)]
    print("Initial population:")
    for c in pop: print(f"  {c}  fitness={tfit(c):>7}  salary={info(c)[1]}k  size={c.count('1')}")
    hist = []
    for g in range(1, gens+1):
        new = [max(pop, key=tfit)]; V = verbose and g <= 2
        if V: print(f"\n--- Generation {g} detail ---")
        while len(new) < pop_n:
            p1, p2 = tour(pop), tour(pop)
            if random.random() < 0.9:
                cp = random.randint(1, n-1); c1, c2 = p1[:cp]+p2[cp:], p2[:cp]+p1[cp:]
            else: cp, c1, c2 = None, p1, p2
            m1 = "".join(("1" if x == "0" else "0") if random.random() < pm else x for x in c1)
            m2 = "".join(("1" if x == "0" else "0") if random.random() < pm else x for x in c2)
            if V:
                print(f"  parents {p1} (f={tfit(p1)}) & {p2} (f={tfit(p2)})  crossover point={cp}")
                print(f"     crossover -> {c1}, {c2}   after mutation -> {m1}, {m2}")
            new += [m1, m2]
        pop = new[:pop_n]; b = max(pop, key=tfit); hist.append(tfit(b))
        print(f"Generation {g:>2}: best={b} fitness={tfit(b):>7}  avg={sum(map(tfit,pop))/len(pop):.1f}")
```

```
    b = max(pop, key=tfit); sel, sal = info(b)
    print(f"\nFinal team {b}: " + ", ".join(e[0] for e in sel))
    print(f"  size={len(sel)}, total salary={sal}k (budget {budget}k), seniors(skill>=8)={sum(e[1]>=8 for e in sel)}, fitness={tfit(b)}")
    return hist, tfit, n

e6 = [("E1",9,8,1.0,120),("E2",6,3,0.8,60),("E3",8,6,0.9,100),("E4",5,2,1.0,40),("E5",7,5,0.6,70),("E6",4,1,1.0,35)]
print("Employees (name, skill, experience, availability, salary k):"); [print("  ", e) for e in e6]
print("Budget=250k, team size 2-4, at least 1 senior (skill>=8). Fitness = sum(3*skill+exp+10*avail) - penalties")
hist, tf, n = team_ga(e6, 250, 2, 4, 1, 20, 30, 4, pm=0.1)
opt = max(("".join(map(str, b)) for b in itertools.product((0,1), repeat=n)), key=tf)
print(f"Exhaustive check over 64 teams: optimum {opt} fitness={tf(opt)}")
```

OUTPUT:

```
Employees (name, skill, experience, availability, salary k):
   ('E1', 9, 8, 1.0, 120)
   ('E2', 6, 3, 0.8, 60)
   ('E3', 8, 6, 0.9, 100)
   ('E4', 5, 2, 1.0, 40)
   ('E5', 7, 5, 0.6, 70)
   ('E6', 4, 1, 1.0, 35)
Budget=250k, team size 2-4, at least 1 senior (skill>=8). Fitness = sum(3*skill+exp+10*avail) - penalties
Initial population:
  010110  fitness=   63.0  salary=170k  size=3
  000110  fitness=   34.0  salary=110k  size=2
  011001  fitness=   91.0  salary=195k  size=3
  001100  fitness=   66.0  salary=140k  size=2
  111011  fitness= -122.0  salary=385k  size=5
  000110  fitness=   34.0  salary=110k  size=2
  101101  fitness=   44.0  salary=295k  size=4
  111100  fitness=    0.0  salary=320k  size=4
  110001  fitness=   97.0  salary=215k  size=3
  111000  fitness=   53.0  salary=280k  size=3
  101101  fitness=   44.0  salary=295k  size=4
  110100  fitness=  101.0  salary=220k  size=3
  010010  fitness=   36.0  salary=130k  size=2
  110010  fitness=  106.0  salary=250k  size=3
  110111  fitness=  -14.0  salary=325k  size=5
  010101  fitness=   54.0  salary=135k  size=3
  101101  fitness=   44.0  salary=295k  size=4
  010101  fitness=   54.0  salary=135k  size=3
  110101  fitness=  114.0  salary=255k  size=4
  000000  fitness=    -65  salary=0k  size=0
```

```
Generation  1: best=011101 fitness=  118.0  avg=76.0
Generation  2: best=011101 fitness=  118.0  avg=69.9
Generation  3: best=011101 fitness=  118.0  avg=87.2
Generation  4: best=011101 fitness=  118.0  avg=87.5
Generation  5: best=011101 fitness=  118.0  avg=66.2
Generation  6: best=011101 fitness=  118.0  avg=57.3
Generation  7: best=011101 fitness=  118.0  avg=87.7
Generation  8: best=011101 fitness=  118.0  avg=78.8
Generation  9: best=011101 fitness=  118.0  avg=88.5
Generation 10: best=011101 fitness=  118.0  avg=66.2
Generation 11: best=011101 fitness=  118.0  avg=82.8
Generation 12: best=011101 fitness=  118.0  avg=85.2
Generation 13: best=001111 fitness=  121.0  avg=96.0
Generation 14: best=001111 fitness=  121.0  avg=82.7
Generation 15: best=001111 fitness=  121.0  avg=90.3
Generation 16: best=001111 fitness=  121.0  avg=80.6
Generation 17: best=001111 fitness=  121.0  avg=89.3
Generation 18: best=001111 fitness=  121.0  avg=94.8
Generation 19: best=001111 fitness=  121.0  avg=68.3
Generation 20: best=001111 fitness=  121.0  avg=102.5
Generation 21: best=001111 fitness=  121.0  avg=69.2
Generation 22: best=001111 fitness=  121.0  avg=75.0
Generation 23: best=001111 fitness=  121.0  avg=71.2
Generation 24: best=001111 fitness=  121.0  avg=106.7
Generation 25: best=001111 fitness=  121.0  avg=91.7
Generation 26: best=001111 fitness=  121.0  avg=92.4
Generation 27: best=001111 fitness=  121.0  avg=93.3
Generation 28: best=001111 fitness=  121.0  avg=88.8
Generation 29: best=001111 fitness=  121.0  avg=77.7
Generation 30: best=001111 fitness=  121.0  avg=67.5

Final team 001111: E3, E4, E5, E6
  size=4, total salary=245k (budget 250k), seniors(skill>=8)=1, fitness=121.0
Exhaustive check over 64 teams: optimum 001111 fitness=121.0
```

- **With a population of 20 and 30 generations the GA finds the best solution (121). A smaller population may stop at 118 without finding the best solution.**

QUESTION NO # 03

Scenario 2 — University Timetable Optimization

A university needs to generate an optimized timetable. A chromosome represents course-to-time-slot assignments. The fitness function should reward conflict-free scheduling, teacher availability, room availability, and balanced scheduling.

Task: Use selection, crossover, and mutation to generate increasingly better timetables. Display the best timetable after every generation.

SOLUTION:

```python
import random, itertools

random.seed(10)
ex = {"C1":("T1",{"G1","G2"},"R1"),"C2":("T2",{"G1"},"R2"),"C3":("T1",{"G3"},"R1"),"C4":("T3",{"G2","G3"},"R2"),
      "C5":("T2",{"G4"},"R1"),"C6":("T4",{"G1","G4"},"R3"),"C7":("T3",{"G4"},"R3"),"C8":("T4",{"G2"},"R2")}
cs = list(ex); SL = ["Mon 9am","Mon 2pm","Tue 9am","Tue 2pm"]; UN = {"T1":{3},"T3":{0}}
def tfit(ch):
    t = dict(zip(cs, ch)); v = 100
    for a, b in itertools.combinations(cs, 2):
        if t[a] == t[b]:
            v -= 10*bool(ex[a][1] & ex[b][1]) + 10*(ex[a][0] == ex[b][0]) + 10*(ex[a][2] == ex[b][2])
    v -= 5*sum(t[c] in UN.get(ex[c][0], set()) for c in cs)
    load = [ch.count(s) for s in range(4)]; v -= 2*(max(load)-min(load))     # balance term
    return v
def show(ch): return " | ".join(f"{SL[s]}: {[c for c in cs if ch[cs.index(c)]==s]}" for s in range(4))
pop = [[random.randrange(4) for _ in cs] for _ in range(30)]
for g in range(0, 16):
    b = max(pop, key=tfit)
    print(f"Gen {g:>2} best fitness={tfit(b):>4}  {show(b)}")
    new = [b[:]]
    while len(new) < 30:
        p1 = max(random.sample(pop, 3), key=tfit); p2 = max(random.sample(pop, 3), key=tfit)
        cp = random.randint(1, len(cs)-1); kids = [p1[:cp]+p2[cp:], p2[:cp]+p1[cp:]]
        for k in kids:
            for i in range(len(k)):
                if random.random() < 0.1: k[i] = random.randrange(4)
        new += kids
    pop = new[:30]
```

OUTPUT:

```
Gen  0 best fitness=  85  Mon 9am: ['C4', 'C5'] | Mon 2pm: ['C1', 'C6'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C7', 'C8']
Gen  1 best fitness=  85  Mon 9am: ['C4', 'C5'] | Mon 2pm: ['C1', 'C6'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C7', 'C8']
Gen  2 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  3 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  4 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  5 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  6 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  7 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  8 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen  9 best fitness=  86  Mon 9am: ['C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C5', 'C6']
Gen 10 best fitness=  90  Mon 9am: ['C1', 'C8'] | Mon 2pm: ['C2', 'C7'] | Tue 9am: ['C3', 'C6'] | Tue 2pm: ['C4', 'C5']
Gen 11 best fitness=  90  Mon 9am: ['C1', 'C8'] | Mon 2pm: ['C2', 'C7'] | Tue 9am: ['C3', 'C6'] | Tue 2pm: ['C4', 'C5']
Gen 12 best fitness= 100  Mon 9am: ['C5', 'C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C6']
Gen 13 best fitness= 100  Mon 9am: ['C5', 'C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C6']
Gen 14 best fitness= 100  Mon 9am: ['C5', 'C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C6']
Gen 15 best fitness= 100  Mon 9am: ['C5', 'C8'] | Mon 2pm: ['C1', 'C7'] | Tue 9am: ['C2', 'C3'] | Tue 2pm: ['C4', 'C6']
```

QUESTION NO # 04

Scenario 3 — Delivery Route Optimization

A courier company has a warehouse and five customers. A chromosome represents the order in which customers are visited, for example A → C → E → B → D.

Use Fitness = 1 / Total Distance. Implement population initialization, fitness calculation, selection, crossover, mutation, and elitism to find a route with minimum total distance.

SOLUTION:

```python
import random, math, itertools

random.seed(10)
P = {"W":(0,0),"A":(2,6),"B":(8,3),"C":(5,9),"D":(9,8),"E":(1,2)}
cust = list("ABCDE"); d = lambda a, b: math.dist(P[a], P[b])
def td(r): p = ["W"]+r+["W"]; return sum(d(p[i], p[i+1]) for i in range(len(p)-1))
ft = lambda r: 1/td(r)
def ox(p1, p2):
    a, b = sorted(random.sample(range(len(p1)), 2)); ch = [None]*len(p1); ch[a:b+1] = p1[a:b+1]
    rest = [g for g in p2 if g not in ch]; return [g if g else rest.pop(0) for g in ch]
pop = [random.sample(cust, 5) for _ in range(12)]
print("Warehouse W + customers", {k: v for k, v in P.items() if k != "W"})
for g in range(0, 13):
    b = max(pop, key=ft)
    print(f"Gen {g:>2}: best route W->{'->'.join(b)}->W  distance={td(b):.3f}  fitness={ft(b):.5f}")
    new = [b[:]]                                          # elitism
    while len(new) < 12:
        p1 = max(random.sample(pop, 3), key=ft); p2 = max(random.sample(pop, 3), key=ft)
        c = ox(p1, p2)
        if random.random() < 0.3: i, j = random.sample(range(5), 2); c[i], c[j] = c[j], c[i]
        new.append(c)
    pop = new
opt = min(itertools.permutations(cust), key=lambda r: td(list(r)))
print(f"\nFinal best: W->{'->'.join(max(pop,key=ft))}->W distance={td(max(pop,key=ft)):.3f}")
print(f"Brute-force optimum (120 routes): W->{'->'.join(opt)}->W distance={td(list(opt)):.3f}")
```

OUTPUT:

```
Warehouse W + customers {'A': (2, 6), 'B': (8, 3), 'C': (5, 9), 'D': (9, 8), 'E': (1, 2)}
Gen  0: best route W->E->A->D->C->B->W  distance=33.015  fitness=0.03029
Gen  1: best route W->E->A->B->D->C->W  distance=32.585  fitness=0.03069
Gen  2: best route W->E->A->B->D->C->W  distance=32.585  fitness=0.03069
Gen  3: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  4: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  5: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  6: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  7: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  8: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen  9: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen 10: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen 11: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525
Gen 12: best route W->E->A->C->D->B->W  distance=28.368  fitness=0.03525

Final best: W->E->A->C->D->B->W distance=28.368
Brute-force optimum (120 routes): W->B->D->C->A->E->W distance=28.368
```